JAVA COURSE

{ getting started }



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1

INDICE

01 Linguaggi di programmazione

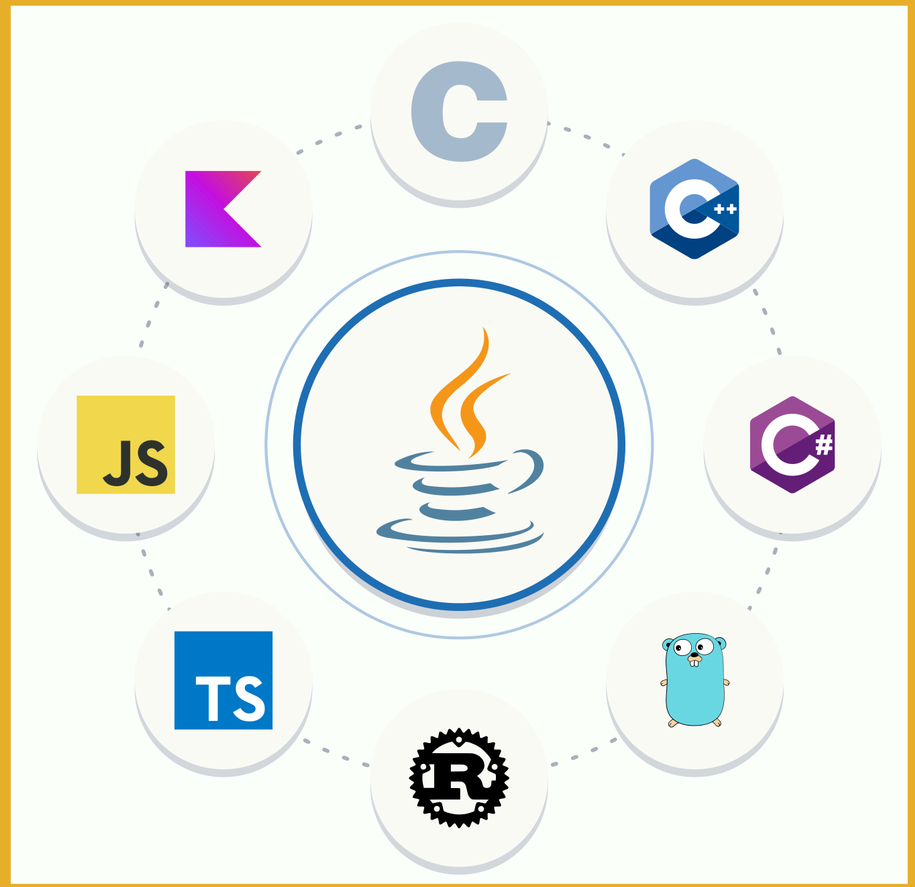
02 La piattaforma

03 Java: storia ed evoluzione

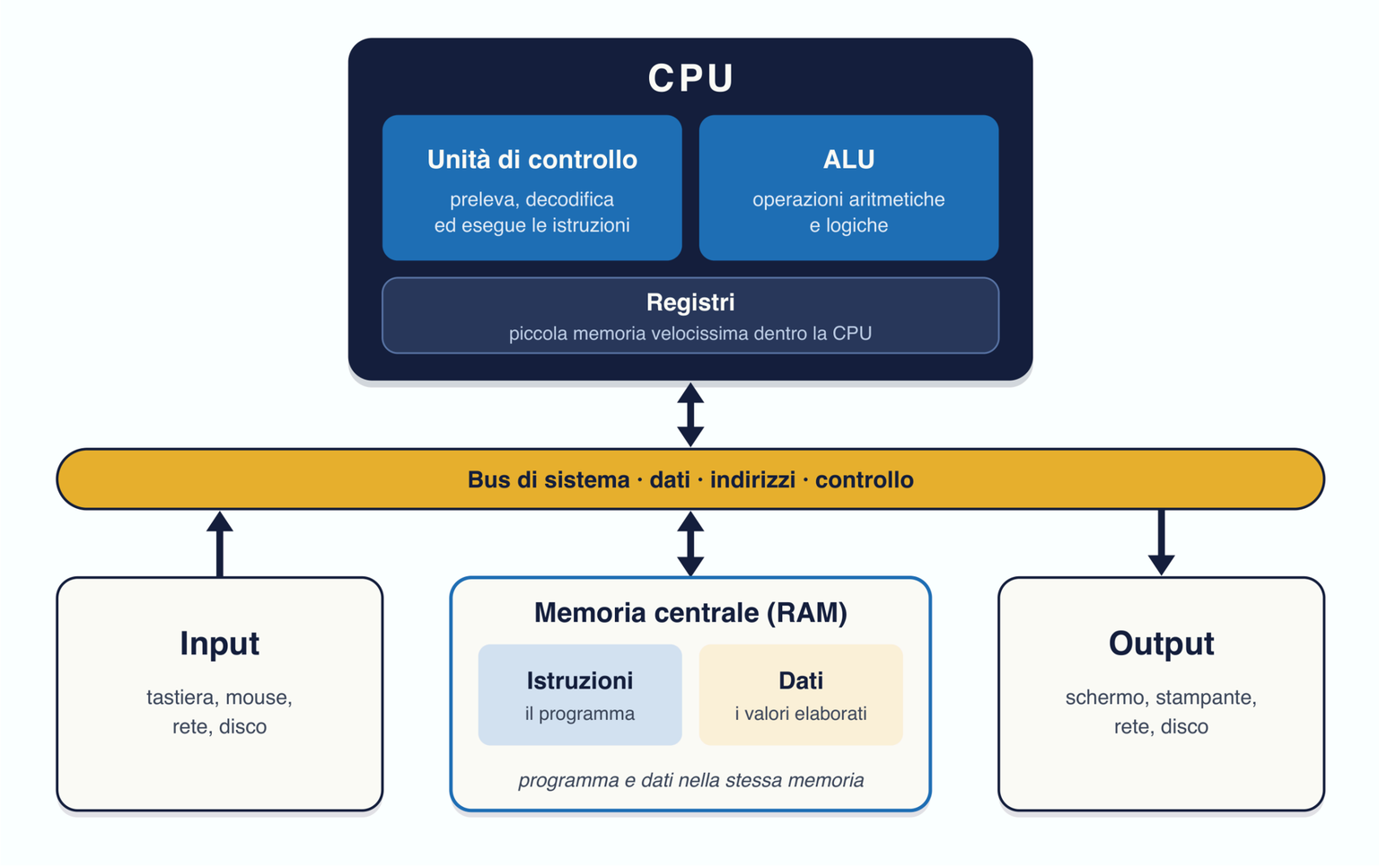
04 Il primo programma: “Hello World!”

01

LINGUAGGI DI PROGRAMMAZIONE



VON NEUMANN


CPU
Unità di controllo
preleva, decodifica
ed esegue le istruzioni
ALU
operazioni aritmetiche
e logiche
Registri
piccola memoria velocissima dentro la CPU
Bus di sistema · dati · indirizzi · controllo
Input
tastiera, mouse,
rete, disco
Memoria centrale (RAM)
Istruzioni
il programma
Dati
i valori elaborati
programma e dati nella stessa memoria
Output
schermo, stampante,
rete, disco

LINGUAGGIO DI PROGRAMMAZIONE

Una **lingua formale** per descrivere algoritmi in modo che una macchina possa eseguirli

- ***Formale:*** *regole precise e senza ambiguità, ogni frase valida ha un solo significato*
- ***Eseguibile:*** *prima o poi va tradotta nelle istruzioni che la CPU sa eseguire*
- ***Per esseri umani:*** *Il codice che si scrive deve essere chiaro per chi lo mantiene*

LINGUAGGI DI PROGRAMMAZIONE E NON

Un linguaggio di programmazione deve poter esprimere qualsiasi algoritmo: in termini tecnici, deve essere **Turing-completo**

- ***Cosa serve:*** *memorizzare dati, prendere decisioni, ripetere operazioni*
- ***HTML:*** *descrive la struttura di una pagina e non esegue calcoli; è un linguaggio di markup*
- ***SQL:*** *interroga i dati; nella forma base non è Turing-completo, lo diventa con le query ricorsive o con estensioni come PL/SQL*

SINTASSI E SEMANTICA

La **sintassi** è la *grammatica* del linguaggio, un errore blocca la traduzione e quindi l'esecuzione; la **semantica** riguarda il *significato* di ogni istruzione, un errore non ne blocca necessariamente l'esecuzione ma il programma fa cose sbagliate

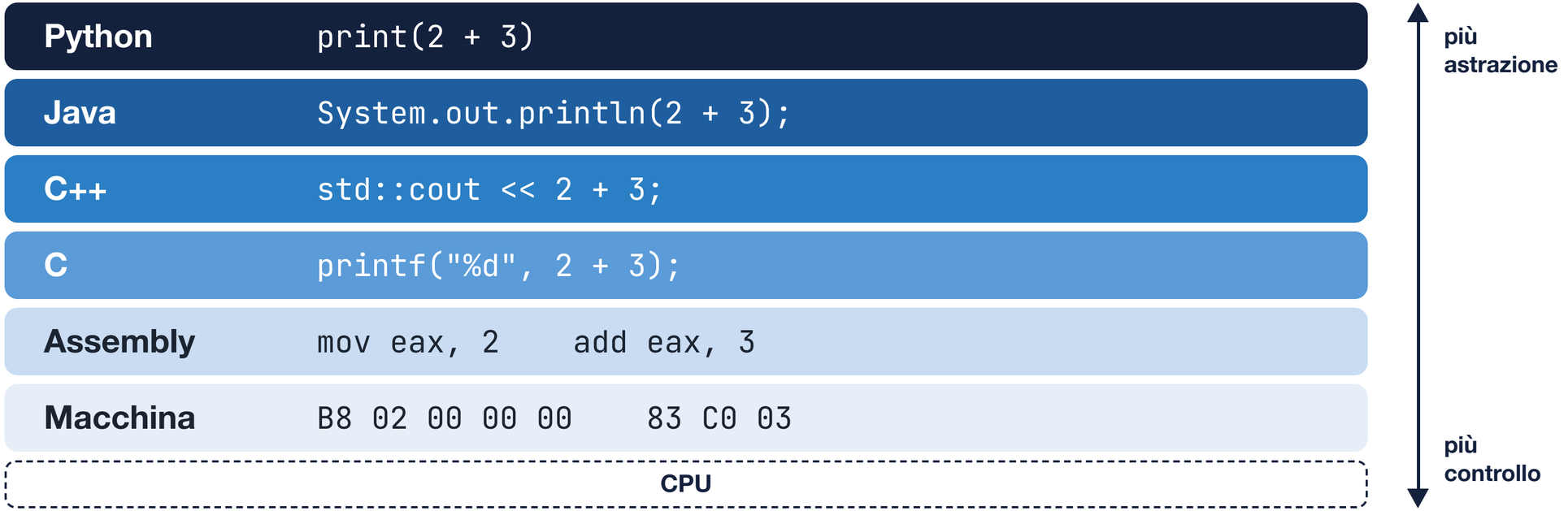
```
int media = (a + b) / 2
// manca il ;
```

```
int media = a + b / 2;
// compila, ma calcola a + (b/2)
```

LA SCALA DEI LIVELLI



LA SCALA DEI LIVELLI



ASSEMBLY E CODICE MACCHINA

L'**assembly** è la forma leggibile del linguaggio macchina: ogni riga corrisponde a un'istruzione della CPU. Un programma chiamato **assembler** la traduce nei byte che la CPU esegue

- ***Uno a uno***
- ***Un assembly per ogni CPU***

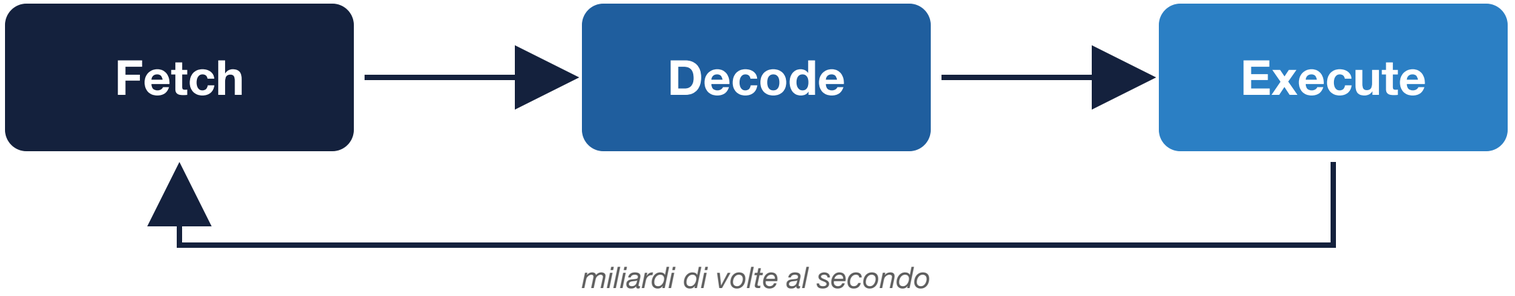
```
mov eax, 2    ; B8 02 00 00 00
add eax, 3    ; 83 C0 03
```

```
mov w0, #2        ; 52800040
add w0, w0, #3    ; 11000C00
```

DOPO LA TRADUZIONE

Il sistema operativo carica il codice macchina in memoria. Da lì la CPU lo esegue un'istruzione alla volta, ripetendo sempre lo stesso ciclo

- ***Fetch***
- ***Decode***
- ***Execute***



IL LINGUAGGIO C

Nasce nei Bell Labs nel 1972, con Dennis Ritchie, per scrivere il sistema operativo Unix. Si compila direttamente in codice macchina

- ***Vicino all'hardware***
- ***Memoria manuale***
- ***Puntatori***

```
int x = 5;
int *p = &x;   // p contiene l'indirizzo di x
*p = 10;       // scrive 10 a quell'indirizzo: ora x vale 10
```

IL LINGUAGGIO C++

Bjarne Stroustrup lo crea nei Bell Labs dal 1979, come "C con le classi": aggiunge al C la programmazione a oggetti

- ***Compatibile con il C***
- ***Classi e oggetti***
- ***Memoria manuale***

```
class Conto {
public:
    double saldo = 0;
    void versa(double importo) { saldo += importo; }
};
```

IL LINGUAGGIO JAVA

Nasce nel 1995 in Sun Microsystems. Riprende la sintassi del C++ e toglie ciò che causa più errori: puntatori espliciti e memoria gestita a mano

- ***Riferimenti, non puntatori***
- ***Garbage collector***
- ***Bytecode e JVM***

```
class Conto {
    double saldo = 0;
    void versa(double importo) { saldo += importo; }
}
```

LINGUAGGIO DI PROGRAMMAZIONE

Ogni linguaggio di alto livello risponde a modo suo a **quattro domande**, utili per confrontarli

1

Come si scrive?

paradigmi

2

Come si esegue?

esecuzione

3

Come controlla i tipi?

tipizzazione

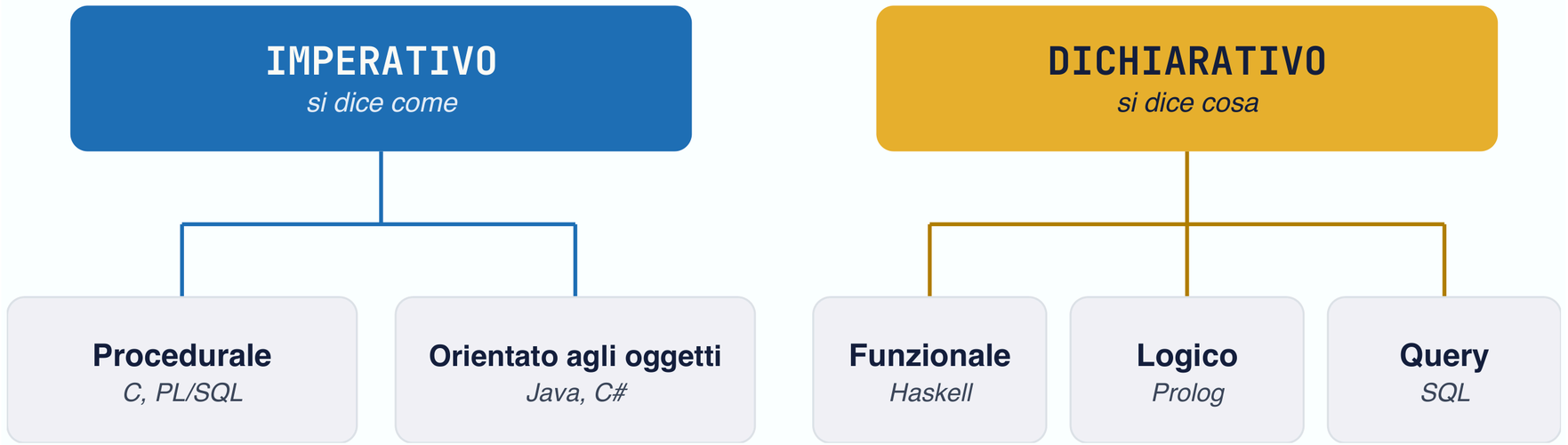
4

Chi gestisce la memoria?

memoria

PARADIGMI DI PROGRAMMAZIONE

Un **paradigma** è uno stile di programmazione: un modo di organizzare il codice e di pensare il problema. Si dividono in due grandi famiglie



IMPERATIVO

Il programma è una **sequenza di istruzioni** che modificano lo stato, passo dopo passo: siamo noi a decidere *come* arrivare al risultato

- ***Procedurale:*** *le istruzioni si raccolgono in procedure riutilizzabili (C, PL/SQL)*
- ***Orientato agli oggetti:*** *dati e comportamento vivono insieme negli oggetti (Java, C#)*

```
int somma = 0;
for (int n : numeri) {
    if (n % 2 == 0) {
        somma += n;
    }
}
```

DICHIARATIVO

Si descrive il **risultato** che si vuole ottenere; *come* ottenerlo lo decide il sistema

- ***Funzionale:*** *funzioni senza effetti collaterali (Haskell)*
- ***Logico:*** *fatti e regole, il sistema deduce (Prolog)*
- ***Query:*** *si interrogano i dati (SQL)*

```
SELECT nome, email
FROM studenti
WHERE anno = 2
ORDER BY nome;
```

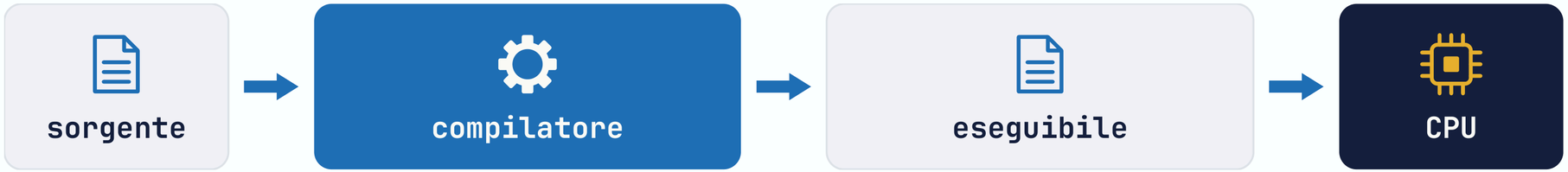
MULTI-PARADIGMA

I linguaggi moderni combinano più paradigmi. Java è nato **a oggetti** e negli anni ha aggiunto altri stili

- ***1995, oggetti:*** *classi, ereditarietà, interfacce*
- ***2014, Java 8:*** *lambda e stream, lo stile funzionale*
- ***2021-23, Java 16-21:*** *record e pattern matching, per programmare orientati ai dati*

IL COMPILATORE

Il codice ad alto livello va sempre tradotto in istruzioni per la CPU. Cambia quando avviene la traduzione



L'INTERPRETE

Il codice ad alto livello va sempre tradotto in istruzioni per la CPU. Cambia quando avviene la traduzione



COMPILATO vs INTERPRETATO

Aspetto	Compilato	Interpretato
Quando si traduce	Prima, una volta	Durante, a ogni esecuzione
Velocità	Di solito più alta	Di solito più bassa
Errori	Molti prima di eseguire	Durante l'esecuzione
Cosa si distribuisce	Un eseguibile per piattaforma	Il sorgente, più l'interprete
Ciclo modifica-prova	Compila, poi esegui	Immediato

COMPILATO vs INTERPRETATO

"*Compilato*" o "*interpretato*" descrive l'implementazione, non il linguaggio

- ***Java*** *si compila in bytecode, che la JVM interpreta e compila con il JIT. Con GraalVM può diventare un eseguibile nativo*
- ***JavaScript*** *è nato interpretato e oggi gira su motori JIT*
- ***Python*** *si compila in bytecode, eseguito dall'interprete CPython; dalla versione 3.13 c'è anche un JIT sperimentale*

BYTECODE e MACCHINA VIRTUALE

Si compila una volta, verso un **codice intermedio** che non dipende dalla CPU, su ogni piattaforma una **macchina virtuale** esegue quel codice

- ***Esempi:*** *Java e Kotlin (JVM), C# (.NET)*



AOT e JIT

AOT (ahead-of-time): tutto il codice diventa codice macchina **prima** dell’esecuzione, avvio immediato e nessun compilatore a runtime

- ***Esempi:*** *C, C++, Rust, Go. Anche Java, con strumenti come GraalVM Native Image*

JIT (just-in-time): il codice viene compilato **durante** l'esecuzione, solo le parti usate più spesso, ottimizzazione in base all’utilizzo

- ***Esempi:*** *JVM, .NET, alcuni motori JavaScript*

TIPIZZAZIONE STATICA vs DINAMICA

Nella tipizzazione **statica** i tipi si controllano **prima** di eseguire. Ogni variabile ha un tipo, dichiarato o dedotto, che non cambia

- *Java, C, C#, Go, Rust, TypeScript*

```
int eta = 30;
eta = "trenta";   // Java: errore di compilazione, String non è int
```

TIPIZZAZIONE STATICA vs DINAMICA

Nella tipizzazione **dinamica** I tipi si controllano **durante** l'esecuzione, sui valori. Una variabile può contenere valori di tipi diversi

- *JavaScript, Ruby, PHP*

```
let eta = 30;
eta = "trenta";   // JavaScript: ok, ora eta contiene una stringa
```

TIPIZZAZIONE DEBOLE vs FORTE

Quanto il linguaggio **converte da solo** un valore da un tipo a un altro. Non c'è una definizione formale unica: è uno spettro. Java converte da solo solo in casi precisi, come l'allargamento dei numeri o la concatenazione con String

```
"3" * 2   // 6
"3" + 2   // "32"
[] + {}   // "[object Object]"
```

```
int n = "3";     // errore
long l = 3;      // ok: int → long
"3" + 2          // "32"
```

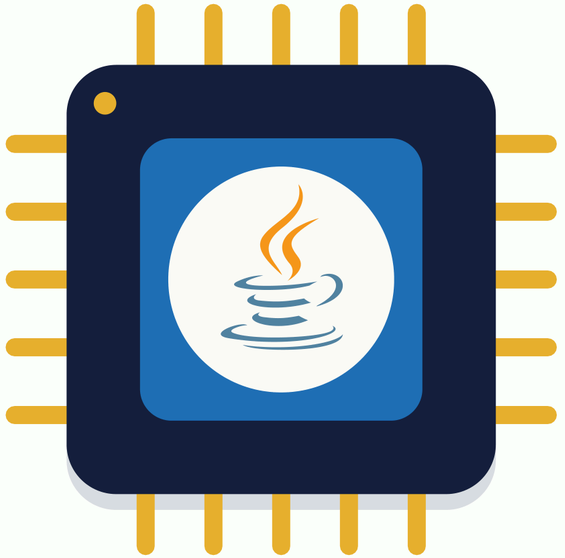
GESTIONE DELLA MEMORIA

Quando un dato non serve più, la sua memoria va liberata. I linguaggi si distinguono per **chi** se ne occupa

- ***In Java:*** *se ne occupa il garbage collector, parte della JVM*

Approccio	Chi libera la memoria	Esempi
Manuale	Il programmatore, con istruzioni esplicite	C, C++
Regole del linguaggio	Il compilatore, in base a chi "possiede" il dato	Rust
Runtime	**Il garbage collector, mentre il programma è in esecuzione**	**Java, C#, JavaScript**

02
LA PIATTAFORMA



0x7FF3A2C0

JAVA SE: LA PIATTAFORMA

Java SE (Java Platform, Standard Edition) è una **specifica**: definisce il linguaggio, la JVM e le librerie standard. Il JDK ne è un'implementazione

- ***La specifica:*** *Java Language Specification, Java Virtual Machine Specification e API*
- ***Chi la definisce:*** *il Java Community Process, con una nuova versione ogni sei mesi*
- ***L'implementazione di riferimento:*** *OpenJDK, da cui derivano tutte le distribuzioni*

JAVAC, IL COMPILATORE

`javac` è il primo "correttore" del nostro codice: se qualcosa non va, il programma non parte nemmeno

- ***Legge:*** *i file sorgente .java*
- ***Controlla:*** *la sintassi e i tipi; se trova errori si ferma e non produce nulla*
- ***Produce:*** *un file .class per ogni classe, con dentro bytecode e non codice macchina*

IL BYTECODE

Istruzioni per una macchina che non esiste come hardware: la **JVM**. Lo stesso `.class` gira su Windows, macOS e Linux, su x86 e su ARM

Terminale

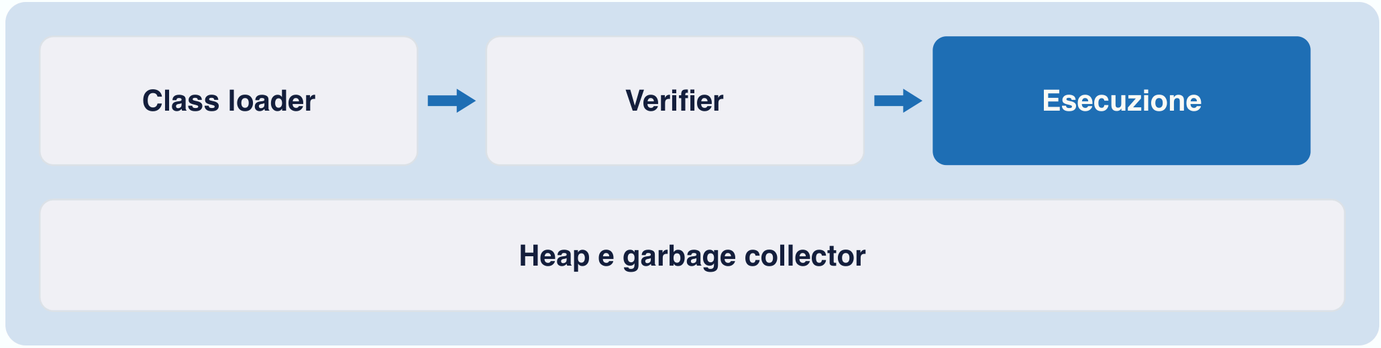
```
$ javac Hello.java
$ ls
Hello.class  Hello.java
$ java Hello
Hello, World!
```

JDK, JRE E JVM


JVM
= JVM
esegue il bytecode
JVM
+
librerie
= JRE
per eseguire
JVM
+
librerie
+
strumenti
= JDK
per sviluppare

DENTRO LA JVM

Il **class loader** carica i .class, il **verifier** controlla che il bytecode sia valido, il **motore di esecuzione** lo esegue. Gli oggetti vivono nello **heap**, ripulito dal garbage collector



I componenti principali della JVM

IL JIT IN QUATTRO PASSI

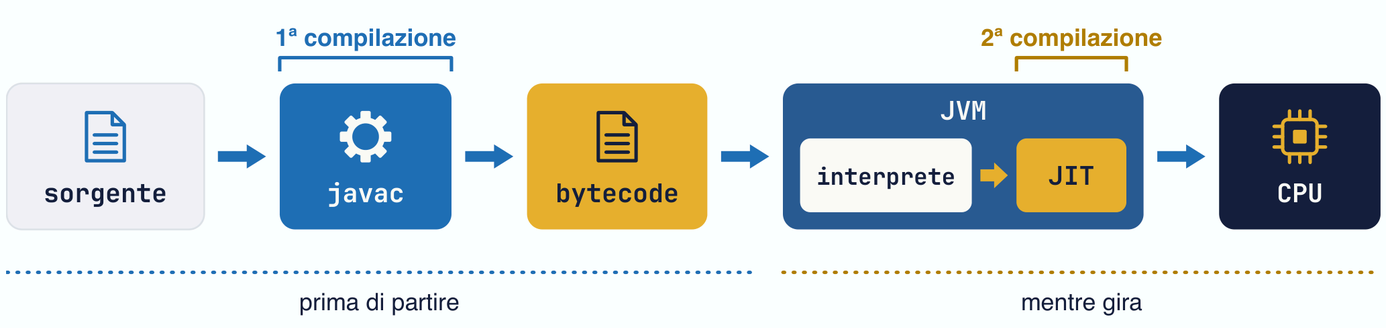
All'avvio la JVM **interpreta** il bytecode, così il programma parte subito. Intanto conta quante volte gira ogni metodo, e quelli usati più spesso li **compila** in codice macchina



Java è insieme "interpretato" e "ad alte prestazioni"

COMPILATO DUE VOLTE

Una volta *prima di partire*, con `javac`: da sorgente a bytecode. Una volta *mentre gira*, con il **JIT**: da bytecode a codice macchina, solo per i metodi usati più spesso



JRE

JRE vuol dire JVM più libreria standard, cioè quello che serve per eseguire.

È cambiato il modo di distribuirlo

- ***Da JDK 11:*** *Oracle non offre più un JRE separato, si installa il JDK*
- ***Altre distribuzioni:*** *alcune, come Temurin, pubblicano ancora pacchetti JRE*
- ***In produzione:*** *l'applicazione porta con sé il suo runtime, creato con jlink o dentro un container*

LA LIBRERIA STANDARD

Il JDK contiene migliaia di classi pronte all'uso, organizzate in `package`.

Prima di scrivere codice, conviene sapere cosa esiste già

- ***Riferimento:*** *la documentazione API di Java 25, su docs.oracle.com*

Package	Contenuto
java.lang	String, Math, System: disponibile senza import
java.util	Collezioni (liste, insiemi, mappe), Scanner, Random
java.time	Date, orari e durate
java.io, java.nio.file	Lettura e scrittura di file
java.math	BigDecimal e BigInteger, per calcoli esatti

JDK

Comando	A cosa serve
javac	Compila i sorgenti in bytecode
java	Avvia la JVM ed esegue un programma, anche direttamente un .java
jshell	Prova codice Java in modo interattivo
javap	Mostra il contenuto di un .class, compreso il bytecode
jar	Impacchetta classi e risorse in un archivio .jar
javadoc	Genera la documentazione dai commenti
jlink	Crea un runtime su misura per un'applicazione

QUALE JDK INSTALLARE

OpenJDK è il codice sorgente; le distribuzioni sono le build pronte da installare. Stessa base, cambiano *licenza*, *supporto* e frequenza degli aggiornamenti

- ***Oracle JDK:*** *è quella del corso*
- ***Open source:*** *Eclipse Temurin, Amazon Corretto, Azul Zulu, Microsoft Build of OpenJDK*

GARBAGE COLLECTOR

In Java gli oggetti si creano con `new` e non si liberano mai a mano: il garbage collector trova quelli che nessuno può più raggiungere e ne recupera la memoria

- ***Il compromesso:*** *meno errori di memoria, in cambio di un po' di lavoro in più per la JVM*

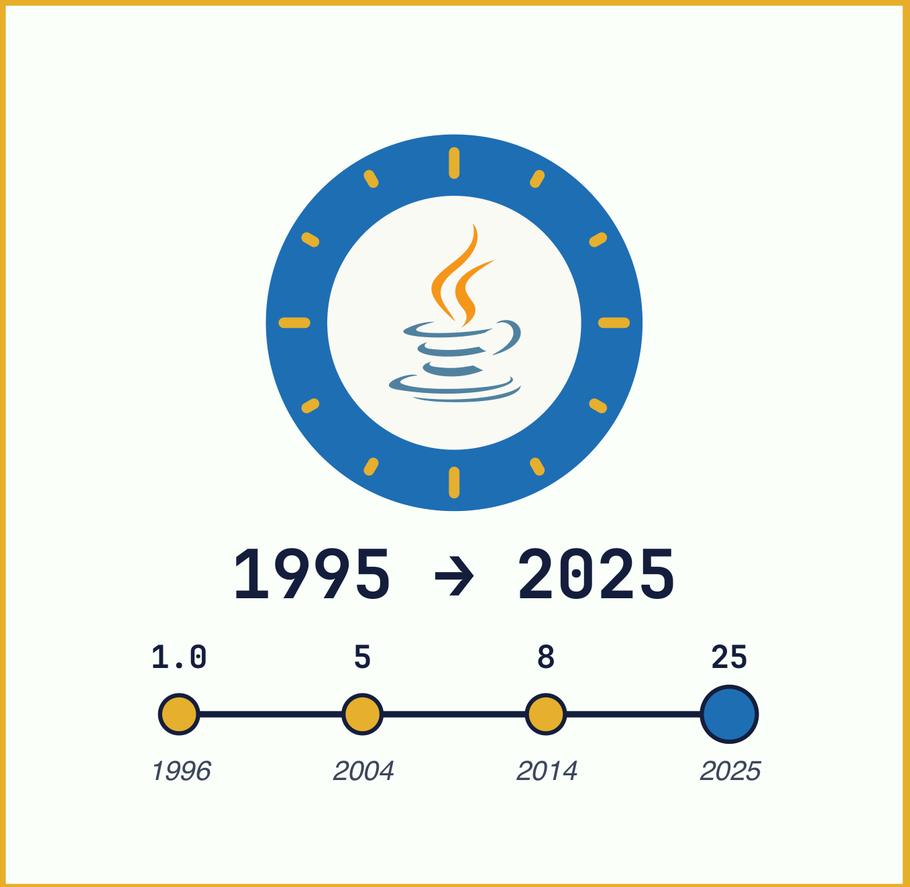
INSTALLARE IL JDK

Nel corso usiamo **Oracle JDK 25**, da scaricare dalla pagina oracle.com/java/technologies/downloads

- ***Verifica:*** *java -version e javac -version devono indicare la versione 25*

Sistema	Pacchetto
Windows	Installer .exe o .msi (x64)
macOS	Installer .dmg: ARM64 per Apple Silicon, x64 per i Mac Intel
Linux	Pacchetto .deb o .rpm, oppure archivio .tar.gz

03
STORIA ED EVOLUZIONE



LE ORIGINI

Nel **1991**, in Sun Microsystems, un piccolo gruppo guidato da **James Gosling** progetta un linguaggio per l'elettronica di consumo, come i decoder per la TV via cavo

- ***Il nome:*** *inizialmente si chiama Oak*
- ***I requisiti:*** *piccolo, affidabile, capace di girare su hardware diversi senza essere riscritto*
- ***L'idea chiave:*** *una macchina virtuale tra il programma e l'hardware*

DAL TELEVISORE AL WEB

Il mercato dei decoder non decolla, ma intanto esplode il **World Wide Web**: pagine lette da computer diversissimi, tutti collegati. È il problema giusto per Java

- ***HotJava:*** *un browser che esegue codice Java dentro le pagine*
- ***Maggio 1995:*** *presentazione pubblica a SunWorld, Oak diventa Java*
- ***Gennaio 1996:*** *esce il JDK 1.0*

LE TAPPE PRINCIPALI

Anno	Tappa
1996	JDK 1.0
1998	Java 1.2: nascono le edizioni SE, EE, ME
2004	Java 5: generics, enum, annotazioni
2006-07	OpenJDK: Java diventa open source
2010	Oracle completa l'acquisizione di Sun
2014	Java 8: lambda e stream
2017-18	Java 9 (moduli), poi una release ogni sei mesi
2025	**Java 25, la LTS di questo corso**

LE EDIZIONI DELLA PIATTAFORMA

Con Java 1.2, nel **1998**, la piattaforma si divide in tre edizioni

- ***SE, Standard:*** *il linguaggio, la JVM e le librerie di base; è quella che studiamo*
- ***EE, Enterprise:*** *specifiche per applicazioni server, come web, persistenza e transazioni*
- ***ME, Micro:*** *dispositivi con poche risorse, come i vecchi cellulari e i sistemi embedded*

DA J2EE A JAKARTA EE

Nel 2018 l'edizione enterprise passa alla Eclipse Foundation e cambia nome. Da Jakarta EE 9 cambiano anche i package: da `javax.*` a `jakarta.*`

```
import javax.persistence.Entity;    // Java EE
import jakarta.persistence.Entity;  // Jakarta EE 9 e successivi
```

OPENJDK

Tra il 2006 e il 2007 Sun pubblica il codice di Java come **OpenJDK**, con licenza GPLv2 e Classpath Exception

- ***2010:*** *Oracle completa l'acquisizione di Sun Microsystems e ne eredita Java*
- ***Oggi:*** *OpenJDK è l'implementazione di riferimento, e tutte le distribuzioni partono da lì*

JAVA 8: LA SVOLTA FUNZIONALE

Nel **2014** arriva il cambiamento più grande del linguaggio dalle origini. Per molti anni Java 8 è stata la versione più usata in azienda

- ***Lambda:*** *pezzi di codice da passare come valori*
- ***Stream API:*** *elaborare collezioni in stile dichiarativo*
- ***java.time:*** *una nuova libreria per date e orari*
- ***Metodi default:*** *interfacce che possono contenere codice*

UNA RELEASE OGNI SEI MESI

Fino a Java 9 le release arrivavano a distanza di anni. Dal **Java 10**, marzo 2018, ne esce una a marzo e una a settembre, con novità più piccole e frequenti

- ***Preview:*** *le funzionalità nuove spesso si provano prima in anteprima, poi diventano definitive*

LE VERSIONI LTS

Alcune release sono **Long-Term Support** e ricevono aggiornamenti per anni; le altre, nel modello Oracle, solo fino alla release successiva. Da Java 17 ne esce una ogni due anni



Organizzazioni che usano la versione in produzione, risposte multiple · Azul, State of Java 2025

QUANTO DURA UNA LTS

Le aziende aggiornano lentamente: in consulenza si trovano ancora progetti su 8, 11 e 17, e molti su 21. Per questo nel corso confrontiamo spesso **21 e 25**

Versione	Uscita	Supporto Premier Oracle fino a
Java 17	settembre 2021	settembre 2026
Java 21	settembre 2023	settembre 2028
Java 25	**settembre 2025**	**settembre 2030**

PERCHÉ JAVA 25

Nel corso usiamo **Java 25**, per tre motivi

- ***È la LTS corrente:*** *supportata per tutta la durata del corso, e oltre*
- ***Spring Boot 4:*** *la supporta, e lo useremo nella seconda parte*
- ***Poche differenze con la 21:*** *per quello che studiamo, e le segnaliamo ogni volta*

LE PAROLE CHIAVE DEL 1995

Il white paper di Sun descriveva Java con undici aggettivi

- *Semplice*
- *Orientato agli oggetti*
- *Distribuito*
- *Robusto*
- *Sicuro*
- *Neutrale rispetto all'architettura*
- *Portabile*
- *Interpretato*
- *Ad alte prestazioni*
- *Multithread*
- *Dinamico*

DOVE SI USA JAVA OGGI

Trent'anni dopo, Java è ovunque ci siano sistemi grandi e longevi. E la JVM ospita anche altri linguaggi: Kotlin, Scala, Groovy, Clojure

- ***Backend enterprise:*** *banche, assicurazioni, pubblica amministrazione, e-commerce*
- ***Android:*** *app scritte in Java o in Kotlin*
- ***Dati e streaming:*** *Kafka, Hadoop ed Elasticsearch girano sulla JVM*
- ***Strumenti:*** *IntelliJ IDEA, Maven, Gradle*

LE CRITICHE A JAVA

Java è anche uno dei linguaggi più criticati. Alcune critiche sono storiche, altre ancora attuali

- ***Verbosità:*** *molto codice per operazioni semplici, ridotto da record e Java 25*
- ***Avvio lento:*** *la JVM impiega tempo a dare il meglio; GraalVM e Leyden lo riducono*
- ***Infrastruttura costosa:*** *serve una JVM ovunque, con più RAM e risorse di un programma nativo*
- ***Ecosistema complesso:*** *molti framework, configurazioni e dipendenze*
- ***Licenze:*** *i cambi di Oracle dal 2019 hanno creato incertezza*

04
IL PRIMO PROGRAMMA

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

```
> Hello, World!
```

HELLO, WORLD!

Per tradizione, il primo programma in un nuovo linguaggio stampa una frase sullo schermo. L'esempio è stato reso celebre nel 1978 dal libro *The C Programming Language*

- ***Lo scopo:*** *verificare che JDK, editor e terminale funzionino insieme*
- ***Cosa mostra:*** *la struttura minima di un programma, la compilazione e l'esecuzione*

LA FORMA CLASSICA

Fino a Java 24 ogni programma deve avere una **classe** e un metodo `main`.

Il file si chiama come la classe: `Hello.java`

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

JAVA 25: VOID MAIN

Da Java 25 un programma può essere scritto in forma compatta, senza dichiarare la classe (JEP 512)

- ***Classe implicita:*** *la genera il compilatore*
- ***IO.println:*** *stampa sulla console*
- ***Import automatici:*** *tutto il modulo java.base*

```
void main() {
    IO.println("Hello, World!");
}
```

JAVA 21 E JAVA 25 A CONFRONTO

La forma classica resta valida ed è quella del codice esistente: nei progetti aziendali è la più diffusa

```java
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

```java
void main() {
    IO.println("Hello, World!");
}
```

COMPILARE ED ESEGUIRE

Due comandi, due strumenti del JDK

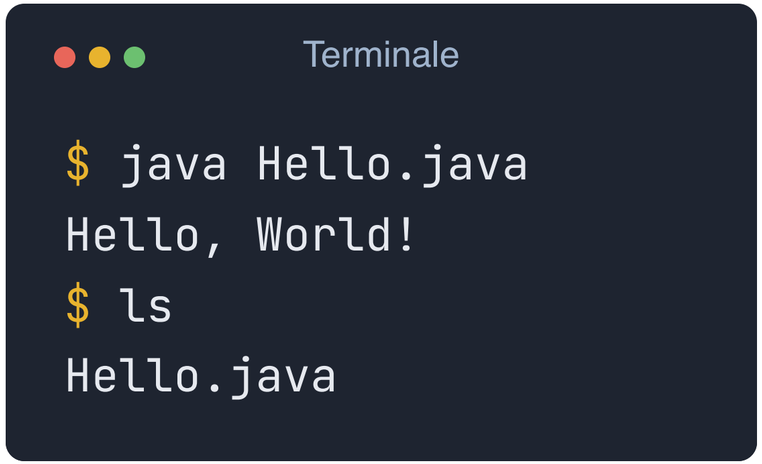
- ***javac Hello.java:*** *compila e produce Hello.class*
- ***java Hello:*** *avvia la JVM con la classe Hello, senza estensione*



ESEGUIRE IL SORGENTE

Dal JDK 11 il comando `java` accetta anche un file sorgente: lo compila in memoria e lo esegue (JEP 330). Dal JDK 22 il programma può essere diviso su più file (JEP 458)

- ***Progetti reali:*** *si compila con javac o con Maven*



QUANDO JAVAC TROVA UN ERRORE

Se il sorgente contiene un errore, javac si ferma e indica dove si trova

- ***Dove:*** *il file e la riga, Hello.java:3; il simbolo ^ indica il punto esatto*
- ***Cosa:*** *manca il punto e virgola a fine istruzione; finché ci sono errori, javac non produce il .class*

Terminale

```
$ javac Hello.java
Hello.java:3: error: ';' expected
        System.out.println("Hello, World!")
                                           ^
1 error
```

JSHELL

Una console interattiva per provare codice Java riga per riga, senza creare file né compilare

- ***Valuta subito:*** *mostra il risultato di ogni espressione*
- ***Adatto a:*** *provare un'istruzione o un metodo della libreria*
- ***/exit:*** *chiude la sessione*

Terminale

```
jshell> int x = 2 + 3;
x ==> 5

jshell> "Hello".length()
$2 ==> 5

jshell> /exit
|  Goodbye
```

I COMMENTI

Il compilatore ignora i commenti: servono a chi legge il codice. I commenti di documentazione diventano pagine HTML con `javadoc`

```
// commento su una riga

/* commento
   su più righe */

/** commento di documentazione (Javadoc) */

/// commento di documentazione in Markdown, da Java 23
```

CONFRONTO CON PYTHON

Gli studenti conoscono Python: ecco le differenze principali

Aspetto	Python	Java
Tipizzazione	dinamica	statica
Compilazione	bytecode `.pyc`, poi interprete	bytecode `.class`, poi JVM con JIT
Esecuzione	`python file.py`	`javac` + `java`, o `java file.java`
Punto di ingresso	`if __name__` (opzionale)	`main` obbligatorio
Blocchi	indentazione	parentesi graffe
Punto e virgola	non serve	obbligatorio

ERRORI COMUNI

I messaggi reali di `javac` e `java`

Errore	Messaggio
Manca il punto e virgola	`';' expected`
Nome del file diverso dalla classe	`class X is public, should be declared in file named X.java`
Classe non trovata	`could not find or load main class`
Errore di battitura nel nome	`cannot find symbol`
Main con firma sbagliata	`Main method not found in class`

DOMANDE?



© 2026 Davide Fella | CC BY-NC 4.0
Uso didattico consentito - V 1.1